

INF221 — Lecture 8

Today's topics—Data structures

1. Operations on dynamic sets
 2. Arrays
 3. Stacks and queues
 4. Linked lists
 5. Lists: implementation details
-

Data Structures

- Manage sets of data that change over time: *dynamic sets*
- Usually distinguish between *key* and *satellite data*
 - We focus here on managing keys
- Compendium, part 11

Operations on dynamic sets

Queries

`Search( $S, k$ )`

Given set S and key k , return reference x to element in S so that $x.key = k$, or `NIL`

`Minimum( $S$ )`

Return reference to the smallest element in S .

`Maximum( $S$ )`

Return reference to the largest element in S .

`Successor( $S, x$ )`

Given a reference x to an element of S , return the next larger element in S , or `NIL` if x is the maximum.

`Predecessor( $S, x$ )`

Given a reference x to an element of S , return the next smaller element in S , or `NIL` if x is the minimum.

Modifying operations

Insert(S, x)

Add the element referenced by x to set S .

Delete(S, x)

Given a reference x to an element in S , remove that element from S .

Notes

- Minimum, Maximum, Successor, and Predecessor are only defined for totally ordered sets S .
- For some data structures, we also may have operations such as Successor that take us through the entire data structure but may return elements in different order on each traversal.
- It is possible to implement a data structure supporting any of the operations above in $O(\lg n)$ for a set of size n (red-black trees).

Dictionaries

A data structure supporting at least Insert, Delete and Search is known as *dictionary*.

Arrays

- Fixed-size arrays are basic but inflexible
- Basic: We can build all more advanced data structures on them
 - First example: heaps
 - Linked-list example later today
- Inflexible:
 - All entries must have same size and type
 - Cannot extend array when it is full
- Solutions:
 - We can *interpret* data in array in different ways (see linked list example)
 - When out of space, copy all data to larger array
 - Place old data at beginning of new array
 - All relations between old data remain the same
 - Space for more elements
 - Usual approach: double array size for each time
 - Can become costly in time and memory for very large arrays

Computer memory can be seen as one huge array: we need to be able to implement all data structures in arrays (at least in principle).

Stacks and queues

- Elementary data structures
- Specific ways of adding and removing elements
- Stack (norsk: "stabel")
 - **LIFO**: last-in, first-out
 - Add and remove elements only from top
 - Common uses: call stack, stack-based calculators, Postscript interpreter
- Queue (norsk: "kø")
 - **FIFO**: first-in, first-out
 - Add new elements at one end, remove elements from other end
 - E.g., customer handling
- Will look at simple implementation in term of arrays here
- Other implementations possible

Stacks

Operations

- Check if stack is empty
- *Push* new element to stack
- *Pop* top element from stack

Implementation

- Fixed-size array S
- `S.length` is the size of the array, i.e., the maximum number of elements on the stack
- Determines maximum number of elements on stack
- `S.top` is index of topmost element on stack

Check for empty

```
Empty(S)
    return S.top == 0
```

Push

```
Push(S, x)
    if S.top == S.length
        error "overflow"
    S.top = S.top + 1
    S[S.top] = x
```

Pop

```
Pop(S)
    if Empty(S)
        error "underflow"
```

```
S.top = S.top - 1
return S[S.top + 1]
```

Implementation as Python class

```
In [1]: class Stack:
        def __init__(self, size):
            self._data = [None] * size
            self._size = size
            self._top = -1

        def empty(self):
            return self._top < 0

        def push(self, x):
            if self._top == self._size - 1:
                raise RuntimeError('Stack overflow')
            self._top += 1
            self._data[self._top] = x

        def pop(self):
            if self.empty():
                raise RuntimeError('Stack underflow')
            self._top -= 1
            return self._data[self._top + 1]

        def top(self):
            if self.empty():
                raise RuntimeError('Stack underflow')
            return self._data[self._top]
```

```
In [2]: S = Stack(3)
        for n in range(3):
            S.push(n**2)

        while True:
            try:
                print(S.pop())
            except RuntimeError as err:
                print(err)
                break
```

```
4
1
0
Stack underflow
```

```
In [3]: S._data
```

```
Out[3]: [0, 1, 4]
```

Implementing a calculator using a stack

- This calculator uses [RPN \(reverse Polish notation\)](#)

- numbers are pushed to stack, operands follow: 3 4 *

```
In [4]: class Calc:
def __init__(self):
    self.stack = Stack(10)

def add(self):
    y = self.stack.pop()
    x = self.stack.pop()
    self.stack.push(x + y)

def sub(self):
    y = self.stack.pop()
    x = self.stack.pop()
    self.stack.push(x - y)

def mul(self):
    y = self.stack.pop()
    x = self.stack.pop()
    self.stack.push(x * y)

def div(self):
    y = self.stack.pop()
    x = self.stack.pop()
    self.stack.push(x / y)

def neg(self):
    x = self.stack.pop()
    self.stack.push(-x)

commands = {'+': add, '-': sub, '*': mul, '/': div, 'neg': neg}

def eval(self, cmd):
    for token in cmd.split():
        try:
            val = float(token)
            self.stack.push(val)
        except ValueError:
            Calc.commands[token](self)
    print(self.stack.top())
```

```
In [5]: mc = Calc()
mc.eval('3 4 +')
```

7.0

$$-((3 + 4) \times 5) + 6$$

```
In [6]: mc.eval('3 4 + 5 * neg 6 +')
```

-29.0

Queues

- Queues have a *head* and a *tail*
- New elements are added to the *tail*
- Elements are removed from the *head*
- Implement using a fixed-size array Q
- "Wrap around" as elements are added and removed
 - Array $Q[1 \dots n]$
 - At most $n - 1$ elements are used
 - $Q.head$ is index of head
 - $Q.tail$ is index for next element to be inserted
 - List elements in $Q.head, Q.head + 1, \dots, Q.tail - 1$
 - $Q.head = Q.tail$ means queue is empty
 - Initially $Q.head = Q.tail = 1$

Algorithms

Insertion: Enqueue

```

Enqueue(Q, x):
    if Q.tail + 1 mod Q.length == Q.head
        error "Queue overflow"
    Q[Q.tail] = x
    if Q.tail == Q.length
        Q.tail = 1
    else
        Q.tail = Q.tail + 1
  
```

Deletion: Dequeue

```

Dequeue(Q):
    if Q.tail == Q.head
        error "Queue underflow"
    x = Q[Q.head]
    if Q.head == Q.length
        Q.head = 1
    else
        Q.head = Q.head + 1
    return x
  
```

Notes

- Both operations are $O(1)$.
- To guard against overflow, we need to take the modulus of $Q.tail+1$ in case that index is wrapping around the end of the array.

Python implementation

```
In [7]: class Queue:
    def __init__(self, size):
        self.size = size
        self._data = [None] * size
        self._head = 0
        self._tail = 0

    def __str__(self):
        return f'Head: {self._head}, Tail: {self._tail}, Data: {self._data}'

    def enqueue(self, x):
        new_tail = (self._tail + 1) % self.size
        if new_tail == self._head:
            raise RuntimeError('Queue is full')
        self._data[self._tail] = x
        self._tail = new_tail

    def dequeue(self):
        if self._head == self._tail:
            raise RuntimeError('Queue empty')
        x = self._data[self._head]
        self._head = (self._head + 1) % self.size
        return x
```

```
In [8]: q = Queue(6)
print(q)
for k in range(4):
    q.enqueue(k**2)
print(q)
```

Head: 0, Tail: 0, Data: [None, None, None, None, None, None]
 Head: 0, Tail: 4, Data: [0, 1, 4, 9, None, None]

```
In [9]: q.enqueue(25)
print(q)
```

Head: 0, Tail: 5, Data: [0, 1, 4, 9, 25, None]

```
In [10]: try:
    q.enqueue(36)
except RuntimeError as err:
    print(err)
```

Queue is full

```
In [11]: print(f"Dequeued: {q.dequeue():4d}, Queue status: {q}")
print(f"Dequeued: {q.dequeue():4d}, Queue status: {q}")
print(f"Dequeued: {q.dequeue():4d}, Queue status: {q}")
```

Dequeued: 0, Queue status: Head: 1, Tail: 5, Data: [0, 1, 4, 9, 25, None]
 Dequeued: 1, Queue status: Head: 2, Tail: 5, Data: [0, 1, 4, 9, 25, None]
 Dequeued: 4, Queue status: Head: 3, Tail: 5, Data: [0, 1, 4, 9, 25, None]

```
In [12]: q.enqueue(v:=36); print(f"Enqueued: {v:4d}, Queue Status: {q}")
q.enqueue(v:=49); print(f"Enqueued: {v:4d}, Queue Status: {q}")
q.enqueue(v:=64); print(f"Enqueued: {v:4d}, Queue Status: {q}")
```

Enqueued: 36, Queue Status: Head: 3, Tail: 0, Data: [0, 1, 4, 9, 25, 36]
 Enqueued: 49, Queue Status: Head: 3, Tail: 1, Data: [49, 1, 4, 9, 25, 36]
 Enqueued: 64, Queue Status: Head: 3, Tail: 2, Data: [49, 64, 4, 9, 25, 36]

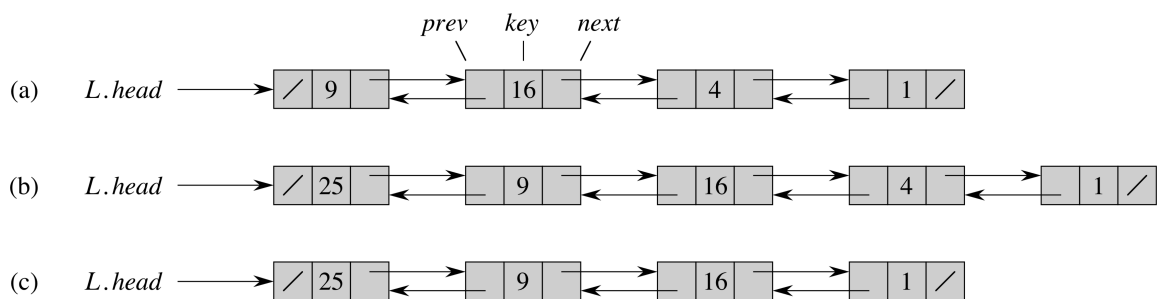
- Note that dequeued data is not deleted/"None"d
- Data stays in place, but head and tail indicate that it is not valid
 - What does this mean for data security?
- We must always have one unused element in the queue
 - Otherwise, `head == tail` could mean both "empty" queue and "full" queue
 - `tail` could no longer simply be interpreted as place of next insertion

Linked lists

- Flexible data structure, especially wrt to insertion and deletion "anywhere"
- Organizes data in a linear fashion
- Search requires traversing all elements until correct key is found
- Singly-linked list
 - Each element points to its successor
 - Traversal only in one direction
- Doubly-linked list
 - Each element points to successor and predecessor
 - Traversal in both directions possible
- *Sorted* list: elements are sorted according to keys
- *Unsorted* list: elements appear in any order, ordering in list is "technical"
- *Circular* list: last element points to first and vice versa

List components

- *Head*: Reference to first element in list
- List elements contain
 - *key*: key value (possibly reference to satellite data, too)
 - *prev*: reference to preceeding element (NIL for first element)
 - *next*: reference to next element (NIL for last element)



(CLRS, Ch 10)

Algorithms

Searching

- Start at head, continue until desired key k found

```
List-Search(L, k)
    x = L.head
    while x != NIL and x.key != k
        x = x.next
    return x
```

- Runtime $\Theta(n)$ in worst case

Insertion

- Insert at front of list
- Need to maintain head, prev and next for involved elements

```
List-Insert(L, x)
    x.prev = NIL
    x.next = L.head
    if L.head != NIL
        L.head.prev = x
    L.head = x
```

- Runtime $O(1)$.

In-class exercise

Extend insertion to insert in between any two elements (useful for sorted lists).

Deletion

- Need to fix up prev and next of surrounding elements
- If first element, need to fix head as well

```
List-Delete(L, x)
    if x.prev != NIL
        x.prev.next = x.next
    else
        L.head = x.next
    if x.next != NIL
        x.next.prev = x.prev
```

- Runtime $O(1)$

Python implementation

- Need two classes
 - Class for the list itself
 - Class for the list items
- We exploit that all variables in Python are references
- The `List` instance only holds the `_head`, pointing to the first `ListItem` in the list
- All other information connecting list elements are in the `ListItems`

```
In [13]: class List:

    class ListItem:
        def __init__(self, key, prv, nxt):
            self.key = key
            self._prev = prv
            self._next = nxt

    def __init__(self):
        self._head = None

    def insert(self, key):
        x = List.ListItem(key, None, self._head)
        if self._head is not None:
            self._head._prev = x
        self._head = x

    def delete(self, x):
        if x._prev is not None:
            x._prev._next = x._next
        else:
            self._head = x._next
        if x._next is not None:
            x._next._prev = x._prev

    def search(self, k):
        x = self._head
        while x is not None and x.key != k:
            x = x._next
        return x
```

```
In [14]: l = List()
```

```
In [15]: l.insert(1)
          l.insert(4)
          l.insert(9)
```

```
In [16]: l.search(9).key
```

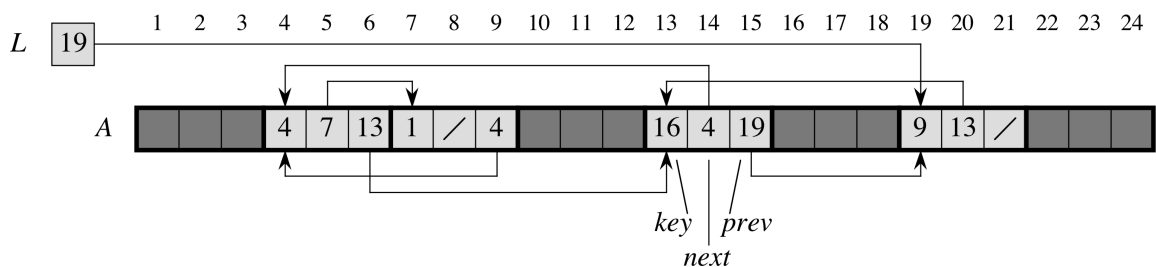
```
Out[16]: 9
```

```
In [17]: print(l.search(10))
```

```
None
```

Lists: "Bare metal" implementation

- Python list implementation above relies on Python "magically" handling references to objects
- Low-level programming languages (C, C++, Java, ...)
 - Use pointers to represent references to objects
 - Pointers are really just memory addresses
 - 0 pointer represents NIL value (invalid pointer)
 - Request memory for individual objects directly from operating system
- Could we create a "bare metal" implementation of a list using just a fixed-size array?



(CLRS, Ch 10)

Bare-metal list in a NumPy array

- Computer memory is just a sequence of bytes
- We assume memory is split into *words*, each word is 8 bytes (64 bits)
- Words are indexed from 0
- We can *imitate* this by a NumPy array with 64-bit integers
- The array represents the memory of our computer

Types and objects

- In our computer, each object consists of a fixed number of contiguous words
- Different "data types" have different numbers of words
- "Data types" determine what the words within an object represent

Data type for list elements

- Fields
 - next
 - prev
 - key
 - data reference
- Each field is one word
 - fields are stored in order shown above
 - prev and next are interpreted as indices into NumPy array

- index -1 is interpreted as NIL
- data reference is memory address of satellite data
- Each object (list entry) thus requires four words or 32 bytes

Handling free memory

- To begin with, all memory is free
- As list elements are added and deleted, "holes" will appear in our array
- Need to be able to recycle those "holes" for new elements
- *Free list*: singly-linked list of all free memory chunks
- Simplifying assumption: all objects are four words long

Metadata

- Head of list
- Head of free list
- For simplicity, we store these outside the NumPy array

Python implementation

```
In [18]: import numpy as np

def show_post(func):
    def wrapped_method(self, *args, **kwargs):
        func(self, *args, **kwargs)
        if self.show:
            self.display()
    return wrapped_method

class AList:
    NIL = -1
    OSZ = 4 # object size in number of array elements
    NEXT = 0 # relative position of fields in object
    PREV = 1
    KEY = 2
    DATA = 3

    def __init__(self, max_entries, show=True):
        self._nmax = max_entries
        self.show = show
        self._mem = np.zeros(AList.OSZ * self._nmax, dtype=np.int64)
        self._head = AList.NIL
        self._free = 0
        # place all words in free list
        for j in range(self._nmax-1):
            self._mem[j * AList.OSZ + AList.NEXT] = (j+1) * AList.OSZ
        # terminate free list with NIL
        self._mem[(self._nmax - 1) * AList.OSZ + AList.NEXT] = AList.NIL

    @show_post
```

```

def insert(self, key, data_id):
    x = self._new_list_item()
    self._mem[x + AList.NEXT] = self._head
    self._mem[x + AList.PREV] = AList.NIL
    self._mem[x + AList.KEY] = key
    self._mem[x + AList.DATA] = data_id

    if self._head != AList.NIL:
        self._mem[self._head + AList.PREV] = x
    self._head = x

@show_post
def delete(self, key):
    x = self._search_item(key)
    pred = self._mem[x + AList.PREV]
    succ = self._mem[x + AList.NEXT]

    if pred != AList.NIL:
        self._mem[pred + AList.NEXT] = succ
    else:
        self._head = succ
    if succ != AList.NIL:
        self._mem[succ + AList.PREV] = pred
    self._free_item(x)

def search(self, key):
    x = self._search_item(key)
    return self._mem[x + AList.DATA]

def _search_item(self, key):
    x = self._head
    while x != AList.NIL and self._mem[x + AList.KEY] != key:
        x = self._mem[x + AList.NEXT]
    if x == AList.NIL:
        raise KeyError
    return x

def _new_list_item(self):
    if self._free == AList.NIL:
        raise MemoryError("Free list empty")
    x = self._free
    self._free = self._mem[self._free + AList.NEXT]
    return x

def _free_item(self, x):
    self._mem[x + AList.NEXT] = self._free
    self._free = x

def display(self):
    s = ['Head' if j == self._head else 'Free' if j == self._free else ' ',
        for j in range(AList.OSZ * self._nmax)]
    t = ['\033[94m####\033[0m'] * self._nmax * AList.OSZ
    x = self._free
    while x != AList.NIL:
        t[x:x+AList.OSZ] = ['\033[91m----\033[0m'] * AList.OSZ
        x = self._mem[x + AList.NEXT]

```

```

print('||' + '||'.join([chr(0xA6).join(['{:4s}']*AList.OSZ)]*self._r
print('||' + '||'.join(['Next|Prev|Key |Data']*self._nmax) + '|')
print('||' + '||'.join([chr(0xA6).join(['{:4d}']*AList.OSZ)]*self._r
print('||' + '||'.join([chr(0xA6).join(['{:4s}']*AList.OSZ)]*self._r

```

In the examples below

- **Free** marks the beginning of the free list
- **Head** marks the beginning of the linked list
- Records are separated by **||**
- ---- indicates free records
- ##### indicates records that are part of the list

In [19]: `al = AList(4)`
`al.display()`

```

|Free|    |    |    ||    |    |    |    ||    |    |    |    ||    |    |
|    |
|Next|Prev|Key |Data|Next|Prev|Key |Data|Next|Prev|Key |Data|Next|Prev|Ke
y |Data|
|  4|  0|  0|  0||  8|  0|  0|  0|| 12|  0|  0|  0|| -1|  0|
0|  0|
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
--|----|

```

In [20]: `al.insert(99, 999)`

```

|Head|    |    |    ||Free|    |    |    ||    |    |    |    ||    |    |
|    |
|Next|Prev|Key |Data|Next|Prev|Key |Data|Next|Prev|Key |Data|Next|Prev|Ke
y |Data|
| -1| -1| 99| 999||  8|  0|  0|  0|| 12|  0|  0|  0|| -1|  0|
0|  0|
|####|####|####|####|----|----|----|----|----|----|----|----|----|----|
--|----|

```

In [21]: `al.insert(88, 888)`

```

|    |    |    |    ||Head|    |    |    ||Free|    |    |    ||    |    |
|    |
|Next|Prev|Key |Data|Next|Prev|Key |Data|Next|Prev|Key |Data|Next|Prev|Ke
y |Data|
| -1|  4| 99| 999||  0| -1| 88| 888|| 12|  0|  0|  0|| -1|  0|
0|  0|
|####|####|####|####|####|####|####|####|----|----|----|----|----|----|
--|----|

```

In [22]: `al.insert(77, 777)`

```

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data|
| -1| 4| 99| 999| | 0| 8| 88| 888| | 4| -1| 77| 777| | -1| 0|
0| 0|
|####|####|####|####| |####|####|####|####| |####|####|####|####| |----|----|----|
--|----|

```

In [23]: `al.insert(66, 666)`

```

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data|
| -1| 4| 99| 999| | 0| 8| 88| 888| | 4| 12| 77| 777| | 8| -1|
66| 666|
|####|####|####|####| |####|####|####|####| |####|####|####|####| |####|####|####|
##|####|

```

In [24]: `al.search(88)`

Out[24]: `np.int64(888)`

In [25]: `try:`
`al.search(55)`
`except KeyError:`
`print('Key 55 not found')`

Key 55 not found

In [26]: `al.delete(88)`

```

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data|
| -1| 8| 99| 999| | -1| 8| 88| 888| | 0| 12| 77| 777| | 8| -1|
66| 666|
|####|####|####|####| |----|----|----|----| |####|####|####|####| |####|####|####|
##|####|

```

In [27]: `al.delete(66)`

```

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data| |Next|Prev|Key |Data|
| -1| 8| 99| 999| | -1| 8| 88| 888| | 0| -1| 77| 777| | 4| -1|
66| 666|
|####|####|####|####| |----|----|----|----| |####|####|####|####| |----|----|----|
--|----|

```

In [28]: `al.insert(55, 555)`

```

|      |      |      |      |Free|      |      |      |      |      |      |      |
|      |
|Next|Prev|Key |Data||Next|Prev|Key |Data||Next|Prev|Key |Data||Next|Prev|Ke
y |Data|
| -1| 8| 99| 999|| -1| 8| 88| 888|| 0| 12| 77| 777|| 8| -1|
55| 555|
|####|####|####|####||---|---|---|---||####|####|####|####||####|####|##
##|####|

```

```
In [29]: al.insert(44, 444)
```

```

|      |      |      |      |Head|      |      |      |      |      |      |
|      |
|Next|Prev|Key |Data||Next|Prev|Key |Data||Next|Prev|Key |Data||Next|Prev|Ke
y |Data|
| -1| 8| 99| 999|| 12| -1| 44| 444|| 0| 12| 77| 777|| 8| 4|
55| 555|
|####|####|####|####||####|####|####|####||####|####|####|####||####|####|##
##|####|

```

```
In [ ]:
```